OpenPoke: Workflow-Centric Agent Orchestration

Addressing Agent Overload Through Persistent Workflows and Intelligent Routing

1. Project Background & Problem Statement

OpenPoke is an open-source multi-agent assistant built around a **Persistent Agent** architecture. Each Execution Agent maintains its own identity and conversation history, enabling users to resume previous tasks across multiple interactions.

However, as users continuously create and interact with agents, the system accumulates an increasing number of persistent agent identities, introducing an **Agent Overload** problem:

- **Context Overload:** The Interaction Agent receives the full historical Agent Roster, causing routing context to grow as more agents are created.
- **Routing Ambiguity:** Multiple agents may share similar names, contacts, or responsibilities, making it increasingly difficult to identify the correct agent for a request.
- **Task-Identity Coupling:** Persistent task context is tightly coupled with agent identity, limiting flexibility in task management and execution orchestration.

These observations motivated a fundamental architectural question:

Do we really need to persist the agent, or should we persist the work that the agent performs?

2. Core Approach: Persistent Workflows + Temporary Execution Agents

To address these limitations, I designed and implemented a **Workflow-Centric Agent Orchestration** architecture that separates long-term task persistence from short-lived agent execution.

The architecture introduces three key components:

- **Persistent Workflow:** Maintains task identity, structured business context, execution history, and lifecycle state.
- **Temporary Execution Agent:** Instantiates an execution runtime on demand while reusing OpenPoke's existing LLM tool-calling capabilities.

- **Intelligent Workflow Routing:** Combines retrieval and LLM-based reasoning to determine whether a request should CREATE, RESUME, or CLARIFY an internal workflow.

The overall architecture follows:

User Request → Hybrid Retrieval → LLM Planner → Deterministic Validator → Workflow Dispatcher → Execution Agent

Importantly, CREATE refers to creating an internal OpenPoke workflow, not automatically creating external business objects such as emails, orders, or support cases.

By decoupling persistent task identity from execution runtime instances, the system enables more scalable task discovery and flexible execution management.

3. System Design & Engineering Implementation

3.1 Persistent Workflow Infrastructure

I implemented a SQLite-backed WorkflowStore to manage persistent workflow metadata, structured context, execution runs, and event history.

Key capabilities include:

- Workflow creation, retrieval, and lifecycle management.
- Persistent execution records and event logging.
- User-level workflow isolation.
- Transactional result persistence.
- Concurrency protection against overlapping runs on the same workflow.

Each workflow is identified by a stable workflow_id, while individual executions receive unique run_id values.

This separation allows task identity and execution history to persist across independently instantiated agent runtimes.

3.2 Hybrid Workflow Retrieval

To eliminate the need to inject the complete historical Agent Roster into every routing request, I developed a hybrid retrieval pipeline combining:

- **BM25 / SQLite FTS5:** Lexical retrieval for workflow titles, keywords, and domain-specific terminology.
- **Dense Retrieval:** Semantic matching using the all-MiniLM-L6-v2 sentence embedding model.
- **Reciprocal Rank Fusion (RRF):** Rank-based fusion of lexical and semantic retrieval results.

The retriever returns a bounded Top-K candidate set containing the workflows most relevant to the current request.

This design reduces the amount of historical task information exposed to the Interaction Agent, making routing context largely independent of the total number of stored workflows.

3.3 LLM Planner + Deterministic Validator

The initial implementation used a rule-based resolver that relied on intent, entity constraints, task categories, and candidate uniqueness.

To improve semantic decision-making, I introduced an **LLM Planner powered by GPT-4.1-mini through OpenRouter**.

The Planner determines one of three routing actions:

- **RESUME:** Continue an existing workflow that uniquely matches the requested task.
- **CREATE:** Create a new internal workflow for an independent or previously untracked task.
- **CLARIFY:** Request additional information when the appropriate workflow cannot be determined unambiguously.

I further refined the Planner's reasoning instructions to emphasize comprehensive candidate comparison, task scope differentiation, and strict interpretation of internal workflow identities.

A separate **Deterministic Validator** enforces programmatic constraints before accepting the Planner's decision, including:

- Candidate workflow membership.
- User ownership and access isolation.

- Workflow status validation.
- Explicit entity and task-type consistency.

This hybrid design combines **LLM-based semantic reasoning with deterministic safety and consistency checks**, rather than allowing the LLM to directly control workflow execution.

3.4 Execution Runtime Integration

I implemented a WorkflowDispatcher and OpenPokeExecutor adapter to connect the new workflow architecture with OpenPoke's original ExecutionAgentRuntime.

This approach preserves the existing multi-turn LLM reasoning loop and tool registry while introducing workflow-based task identity and execution lifecycle management.

Integration tests verified:

- Workflow creation and continuation.
- Persistent execution history restoration.
- Cross-workflow context isolation.
- Multi-turn LLM tool calling with mocked external services.
- Execution state persistence and error handling.

The implementation extends the existing OpenPoke architecture rather than replacing its core execution capabilities.

4. Evaluation Methodology & Experimental Results

To evaluate the proposed architecture, I developed a reproducible evaluation framework covering retrieval quality, routing accuracy, context scalability, and execution integration.

The V2 benchmark contains **200 stored workflows and 200 labeled synthetic requests**, divided into a 40-case development set and a 160-case holdout set.

The dataset includes exact-reference matching, semantic task continuation, new tasks involving existing contacts, ambiguous workflow identities, missing internal workflow references, and multi-workflow requests.

4.1 Routing Accuracy

The final evaluation used 200 stored workflows and 160 labeled routing requests.

Metric	Rule-based Resolver	Final LLM Planner + Validator
Routing Accuracy	46.88%	**98.75%**
Correct Resume Rate	42.65%	**98.53%**
Correct Routing Decisions	75/160	**158/160**
Incorrect Resume Decisions	0	**1**

The final LLM Planner + Validator improved routing accuracy by 51.87 percentage points compared with the rule-based baseline.

During prompt refinement, routing accuracy increased from 95.63% to 98.75%, while incorrect workflow resumptions decreased from five to one.

The improvements were primarily associated with better disambiguation among similar workflows, more precise task-scope reasoning, and stricter interpretation of internal workflow identity.

These results demonstrate the value of semantic planning when deterministic routing rules become overly restrictive or insufficiently expressive.

4.2 Routing Context Scalability

I also measured routing-context size using the V2 dataset with 200 historical workflows.

Approach	Routing Context Size
Original Full Agent Roster	9,902 characters
Workflow Top-5 Retrieval	1,137 characters on average
Context Reduction	**88.52%**

By replacing the full historical roster with a bounded set of relevant workflow candidates, the architecture reduces the growth of routing context as the number of stored workflows increases.

This directly addresses the **Routing Context Overload** problem identified in the original OpenPoke architecture.

The reported reduction represents routing-section character length, rather than total LLM token usage.

4.3 Runtime Performance

The final V2 evaluation produced the following latency measurements:

Component	P50 Latency
Hybrid Retrieval	~26 ms
Rule-based Resolver, including retrieval	~50 ms
LLM Planner, using precomputed candidates	~1.36 s

All 160 live Planner API calls returned token usage records, with no recorded planner fallbacks.

These measurements illustrate the trade-off between lightweight deterministic routing and the improved semantic decision quality provided by an LLM Planner.

5. Key Contributions & Engineering Impact

The primary contribution of this project is not simply replacing rule-based routing with an LLM. It is the architectural separation of **persistent task state, intelligent workflow resolution, and on-demand agent execution**.

1. Scalable Context Management

Introduced bounded Top-K workflow retrieval, reducing routing-context character length by 88.52% in the V2 benchmark and avoiding linear growth from injecting the complete historical roster.

2. Improved Semantic Routing

Combined Hybrid Retrieval, LLM Planning, and Deterministic Validation to achieve 98.75% routing accuracy on 160 labeled evaluation requests.

3. More Reliable Workflow Identification

Improved candidate comparison, ambiguity handling, and workflow identity reasoning, reducing incorrect resume decisions from five to one during prompt refinement.

4. Decoupled Persistence and Execution

Separated long-term workflow identity from execution runtime instances, enabling persistent task continuity without requiring the task itself to be represented by a permanent agent instance.

5. Modular and Testable Architecture

Designed independent components for retrieval, planning, validation, dispatching, persistence, and execution, allowing each subsystem to be tested and optimized separately.

6. Reproducible Evaluation

Established benchmark datasets, automated evaluation scripts, integration tests, and structured experiment reports to support evidence-based architectural decisions.

6. Current Scope & Future Improvements

The current prototype implements persistent workflow management, real-LLM workflow routing, deterministic decision validation, and an execution integration path tested with mocked external tools.

The remaining routing errors are concentrated in exact-reference retrieval and ambiguity among highly similar workflows.

Future improvements include:

- Adding structured exact-identifier lookup alongside Hybrid Retrieval.
- Strengthening ambiguity detection and task-scope validation.
- Supporting requests spanning multiple independent workflows.
- Integrating Planner decisions into the existing Interaction Agent LLM call to reduce additional inference latency.
- Extending fault recovery and idempotency safeguards for real external tool operations.

The current routing results were obtained on synthetic requests with annotated intent and entity hints. The final prompt refinement was evaluated on the same previously inspected holdout set; therefore, additional unseen-data evaluation would further establish generalization performance.

7. Conclusion

This project addresses OpenPoke's Agent Overload problem by introducing a **Workflow-Centric Agent Orchestration** architecture built around persistent workflows, intelligent retrieval, LLM-guided routing, and temporary execution runtimes.

Rather than maintaining persistent agent identities as the primary representation of long-running tasks, the system treats workflows as durable units of work and agents as execution resources.

The final V2 experiments demonstrated:

- **88.52% reduction in routing-context character length.**
- **98.75% workflow routing accuracy.**
- **98.53% correct workflow resume rate.**
- **80% reduction in incorrect resume decisions following prompt refinement.**

Together, these results provide evidence that workflow-centric orchestration can improve routing-context scalability and semantic task resolution while enabling more modular execution lifecycle management.

Core Design Principle

Persist the Work, Not the Worker.